

Analysis of BGC content across phylum Cyanobacteriota

Setup

Read in data

```
# Map antiSMASH classes to their categories
bgc_class <- read_tsv("data/2025-01-16-1256-bgc_class_ref.tsv") %>% select(!owner_id)
class_to_cat <- bgc_class$bgc_category
names(class_to_cat) <- bgc_class$class_name

# From SMC: All antiSMASH 'region's for Cyanobacteriota genomes
regions_unfiltered <- read_tsv("data/2025-02-26-1456-cyano_as_regions.tsv")

# NCBI Taxonomy data for Cyanobacteriota assemblies
# assembly accession and selected tax ranks
# 6.1k entries
cyano_asm_tax <- read_tsv("data/cyano_asm_tax.tsv")
# assembly accession, assembly level, taxid, and select tax ranks
# 10.2k entries
cyano_asm_dataset <- read_tsv("data/cyanos_genomes_taxids.tsv") %>%
  left_join(
    read_tsv("data/taxonomy_summary.tsv"),
    by = join_by("Organism Taxonomic ID" == "Taxid")
  ) %>%
  select(
    `Assembly Accession`,
    `Assembly Level`,
    `Organism Taxonomic ID`,
    `Superkingdom name`,
    `Kingdom name`,
    `Phylum name`,
    `Class name`,
    `Order name`,
    `Family name`,
    `Genus name`,
    `Species name`
  )

# Table of NCBI cyano assemblies at "chromosome" or "complete" quality levels
ncbi_hiq_meta <- read_tsv("data/ncbi_cyano_HiQualityGenomes_metadata.tsv")

# List of the accessions that have been "seen" by the SMC BGC pipeline, whether BGCs were found or not
# NOTE: Does not include accessions that were found to be MD5-identical (I think)
smc_track_df <- read_tsv("data/smc_tracked_accessions.tsv") %>%
```

```

pivot_longer(
  cols = ends_with("Accession"),
  names_to = "source",
  names_pattern = "([[:alnum:]]+)Accession",
  values_to = "accession"
) %>%
drop_na(accession)

smc_tracked_cyanos_df <- smc_track_df %>%
  semi_join(ncbi_hiq_meta, by = join_by(accession == `Assembly Accession`)) %>%
  inner_join(cyano_asm_dataset, by = join_by(accession == `Assembly Accession`)) %>%
  mutate(id_sfx = str_remove(accession, "GC[FA]")) %>%
  arrange(id_sfx)

# BiG-SLiCE results for getting GCF counts per genus
bigslice_df <- read_tsv("data/2025-03-14-1524-bigslice_results.tsv") %>%
  left_join(
    cyano_asm_dataset,
    by = join_by("orig_folder" == "Assembly Accession")
  ) %>%
  rename(
    "asm_level" = `Assembly Level`,
    "taxid" = `Organism Taxonomic ID`,
    "superkingdom" = `Superkingdom name`,
    "kingdom" = `Kingdom name`,
    "phylum" = `Phylum name`,
    "class" = `Class name`,
    "order" = `Order name`,
    "family" = `Family name`,
    "genus" = `Genus name`,
    "species" = `Species name`
  ) %>%
  replace_na(list(genus = "Unclassified"))

# Handy vector of accession suffixes (i.e. remove the GCA/GCF from assembly
# accession) for RefSeq/GenBank assemblies that are identical to each other

# If a RefSeq assembly accession is present in one dataset, but only the
# GenBank assembly was analyzed for another dataset, joining the two datasets
# can retain them both

# filter huge file for accessions found in cyano-specific `ncbi_hiq_meta`
ident_accession_sfx <- read_tsv("data/assembly_summary_genbank.txt", skip = 1) %>%
  rename(assembly_accession = `#assembly_accession`) %>%
  select(
    assembly_accession,
    gbrs_paired_asm,
    paired_asm_comp,
    assembly_level
  ) %>%
  filter(

```

```

assembly_level %in% c("Complete Genome", "Chromosome"),
paired_asm_comp == "identical"
) %>%
semi_join(
  ncbi_hiq_meta,
  by = join_by(assembly_accession == `Assembly Accession`)
) %>%
pull(gbrs_paired_asm) %>%
str_remove("GC[AF]")

```

Clean data

```

# Convert BGC 'class' to vector, add in BGC 'category' as vector, order levels
# of classes and categories
regions_unfiltered <- regions_unfiltered %>%
  mutate(classes = map(region_class, function(class_string) {
    # convert a JSON-string list of BGC classes to an R vector
    # "["NRPS", 'PKS']" -> c('NRPS', 'PKS')
    # if it's just one class, it's not JSON-ified already
    # "NRPS" -> c('NRPS')
    (
      if (str_starts(class_string, fixed("["))) {
        fromJSON(class_string)
      } else {
        c(class_string)
      }
    )
  })) %>%
  mutate(categories = classes %>% map(function(cls_vec) {
    # map the BGC classes to their categories and condense
    (map_vec(cls_vec, function(cls_str) class_to_cat[[cls_str]]) %>% unique() %>% sort())
  })) %>%

  mutate(cats_str = categories %>% map_chr(function(x) str_flatten(x, collapse = ", "))) %>%
  add_count(cats_str) %>%
  mutate(
    cats_str = forcats::fct_reorder(cats_str, desc(n)),
    class_str = classes %>% map_chr(function(x) str_flatten(x, collapse = ", "))
  )

# use the vector of identical accession suffixes to deduplicate the datasets
regions_unfiltered <- regions_unfiltered %>%
  mutate(
    id_sfx = str_remove(accession_id, "GC[AF]"),
    pair_is_ident = (id_sfx %in% ident_accession_sfx)
  )

bigslice_df <- bigslice_df %>%
  mutate(
    id_sfx = str_remove(orig_folder, "GC[AF]"),
    pair_is_ident = (id_sfx %in% ident_accession_sfx)
  )

```

```

smc_tracked_cyanos_df <- smc_tracked_cyanos_df %>%
  mutate(pair_is_ident = (id_sfx %in% ident_accession_sfx))

# For accessions seen by SMC, if they are identical to their pair but the pair
# accession has not been seen, then add the missing rows to the tracker,
# swapping GCF <-> GCA as needed

smc_tracked_cyanos_df <- bind_rows(
  smc_tracked_cyanos_df,
  smc_tracked_cyanos_df %>%
    filter(n() == 1, pair_is_ident == TRUE, .by = id_sfx) %>%
    mutate(accession = case_when(
      source == "Genbank" ~ str_replace(accession, "GCA", "GCF"),
      source == "Refseq" ~ str_replace(accession, "GCF", "GCA")
    ))
)

```

Explore data

The repetitive composition of many BGCs makes them a challenge during genome assembly, resulting in over-inflation of BGC counts when BGCs are split between the ends of two different contigs. Focusing on high-quality genomes can therefore ensure a higher-quality dataset.

Basic summary

There are 4090 BGC-containing genomes and 32112 BGCs in the initial dataset.

How fragmented are the full set of genomes, and how does that impact BGC counts?

```

regions_unfiltered %>%
  group_by(smc_id, n_scaffolds, contig_edge) %>%
  summarize(n_bgcs = n()) %>%
  ungroup() %>%
  ggplot(aes(x = n_scaffolds, y = n_bgcs)) +
  stat_bin_hex(bins = 50) +
  scale_x_log10(breaks = breaks_log()) +
  guides(fill = guide_colorbar(title = "# Genomes")) +
  facet_wrap(
    . ~ contig_edge,
    ncol = 1,
    labeller = as_labeller(c(
      "FALSE" = "BGCs not on contig edge",
      "TRUE" = "BGCs on contig edge"
    ))
  ) +
  theme_bw() +
  ggtitle(
    "Fragmented genomes have inflated BGC counts",
  )

```

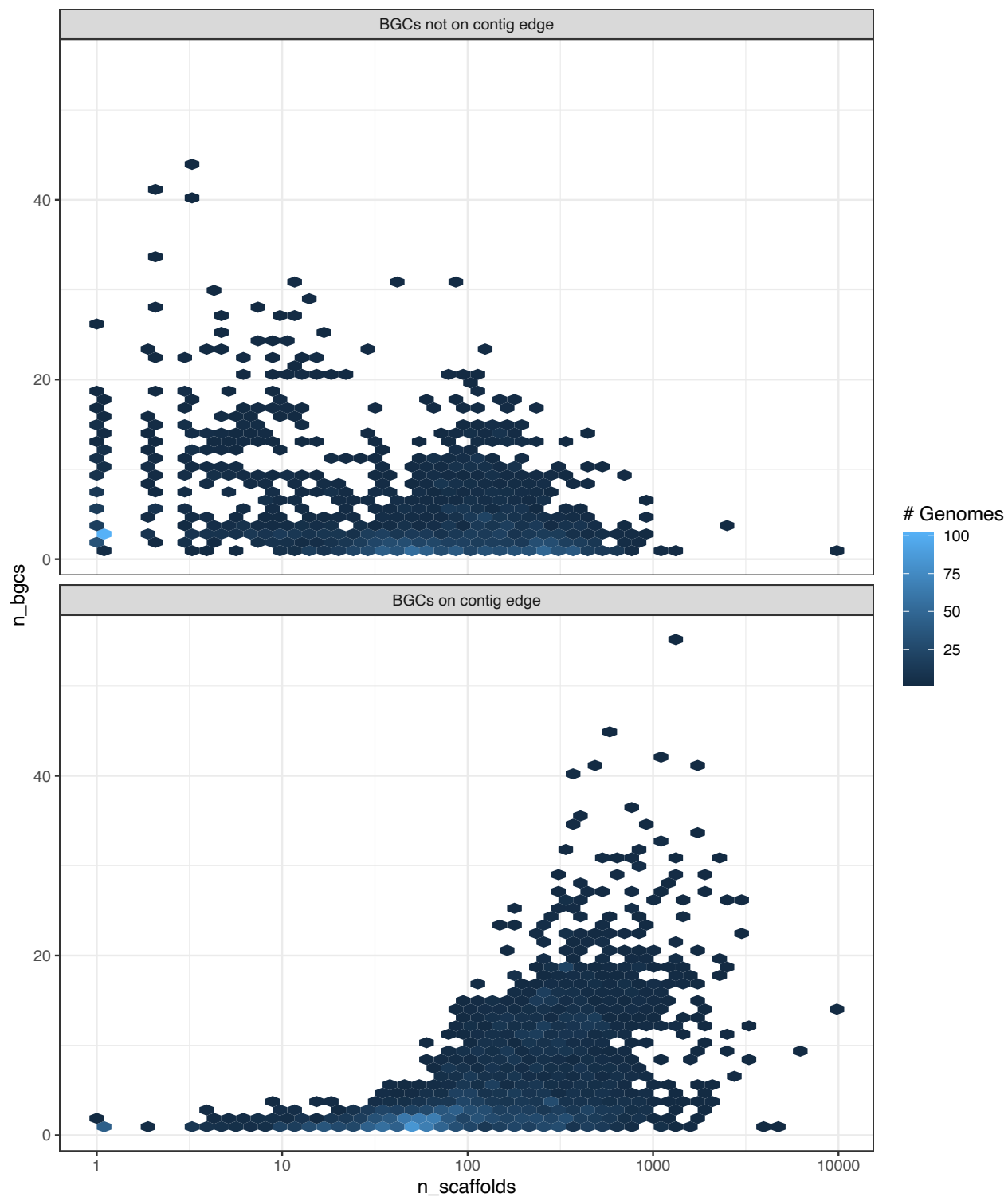
```

subtitle = "Full dataset"
)

```

Fragmented genomes have inflated BGC counts

Full dataset



This figure depicts the number of BGCs against the number of scaffolds in a genome. To help avoid over-

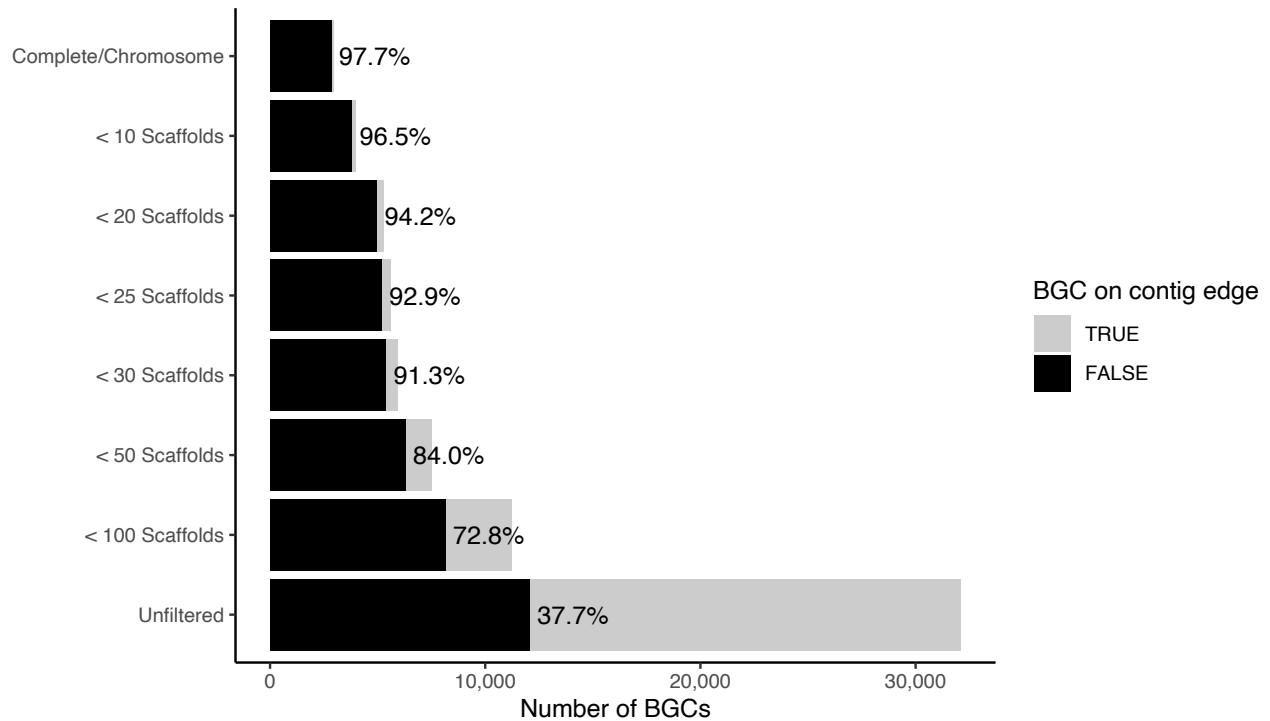
plotting (i.e. many overlapping data points misrepresenting the distribution of the data), the colors of each spot in the figure correspond to how many data points overlap at those coordinates.

How does the proportion of BGCs off/on a contig edge change if we filter for high-quality genomes in different ways?

```
filters_df <- bind_rows(
  regions_unfiltered %>%
    group_by(contig_edge) %>%
    summarize(filter = "Unfiltered", n = n()) %>%
    ungroup() %>%
    mutate(pct = 100 * n / sum(n)),
  regions_unfiltered %>%
    semi_join(ncbi_hiq_meta, by = join_by(accession_id == `Assembly Accession`)) %>%
    group_by(contig_edge) %>%
    summarize(filter = "Complete/Chromosome", n = n()) %>%
    mutate(pct = 100 * n / sum(n)),
  regions_unfiltered %>%
    filter(n_scaffolds < 10) %>%
    group_by(contig_edge) %>%
    summarize(filter = "< 10 Scaffolds", n = n()) %>%
    mutate(pct = 100 * n / sum(n)),
  regions_unfiltered %>%
    filter(n_scaffolds < 20) %>%
    group_by(contig_edge) %>%
    summarize(filter = "< 20 Scaffolds", n = n()) %>%
    mutate(pct = 100 * n / sum(n)),
  regions_unfiltered %>%
    filter(n_scaffolds < 25) %>%
    group_by(contig_edge) %>%
    summarize(filter = "< 25 Scaffolds", n = n()) %>%
    mutate(pct = 100 * n / sum(n)),
  regions_unfiltered %>%
    filter(n_scaffolds < 30) %>%
    group_by(contig_edge) %>%
    summarize(filter = "< 30 Scaffolds", n = n()) %>%
    mutate(pct = 100 * n / sum(n)),
  regions_unfiltered %>%
    filter(n_scaffolds < 50) %>%
    group_by(contig_edge) %>%
    summarize(filter = "< 50 Scaffolds", n = n()) %>%
    mutate(pct = 100 * n / sum(n)),
  regions_unfiltered %>%
    filter(n_scaffolds < 100) %>%
    group_by(contig_edge) %>%
    summarize(filter = "< 100 Scaffolds", n = n()) %>%
    mutate(pct = 100 * n / sum(n)),
)

ggplot(filters_df, aes(x = filter, y = n)) +
  geom_col(aes(fill = fct_rev(as_factor(contig_edge))), position = position_stack()) +
  geom_text(aes(y = n, label = sprintf("%1.1f%%", pct)), data = filters_df %>% filter(contig_edge == FA
```

```
scale_x_discrete(name = "", limits = c("Unfiltered", "< 100 Scaffolds", "< 50 Scaffolds", "< 30 Scaffolds", "Complete/Chromosome"), labels = label_comma()) +
scale_y_continuous(name = "Number of BGCs", labels = label_comma()) +
scale_fill_manual(name = "BGC on contig edge", values = c("gray80", "black")) +
theme_classic() +
coord_flip()
```



This figure depicts the counts of BGCs that are on a contig edge vs. those that are not, depending on how we define what a “high-quality genome” is. Complete/Chromosome refers to the genomes at the “Complete” or “Chromosome” assembly levels on NCBI.

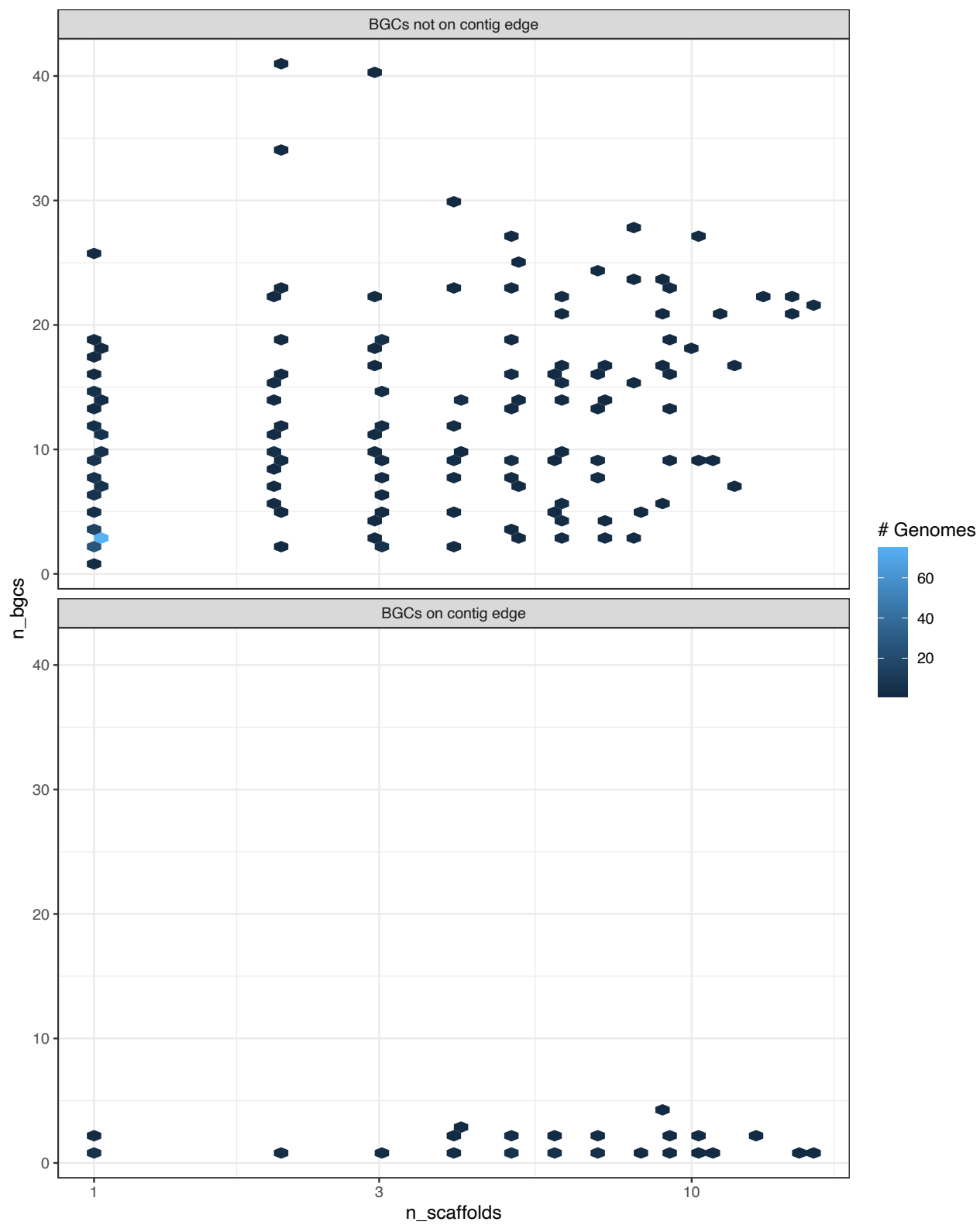
Based on this figure, and to be most conservative in this analysis, we will be going with the most restrictive criteria – using only genomes of “Chromosome” or “Complete” assembly quality as listed on NCBI.

Filter the dataset to high-quality genomes

Repeat the figure from above, and we should see that most BGCs are not on a contig edge.

```
regions <- regions_unfiltered %>%
  semi_join(ncbi_hiq_meta, by = join_by(accession_id == `Assembly Accession`))
```

Now, there are 326 genomes and 2953 BGCs represented in the dataset.



Analyze data

Now we will proceed with our analysis, with the goal of looking at the BGC content of phylum Cyanobacteriota across the axes of length, BGC category, and taxonomy.

Note: AntiSMASH-annotated BGCs are assigned one or more of several dozen BGC “classes” based on the detection rule(s) triggered. These classes can also be grouped into one of 7 “categories” as defined by MIBiG – namely Polyketide, NRP, RiPP, Terpene, Saccharide, Alkaloid, and Other.

Summary statistics of BGC length across BGC categories

```
## # A tibble: 22 x 7
## # Groups:   cats_str [22]
##   cats_str          n min_len max_len mean_len median_len      sd
##   <fct>         <int>   <dbl>   <dbl>   <dbl>     <dbl>   <dbl>
## 1 Terpene         834   13541   39454   20403.    20612.   2079.
## 2 RiPP            739    7553   76778   29623.    24133   10016.
## 3 NRP             500   20873   97395   47693.    43942.  10078.
## 4 Polyketide      307   22218   87362   46966.    46241    6996.
## 5 NRP, Polyketide  296   15761  190414   73530.    70899  24984.
## 6 Other           97   10034   60591   23411.    20762  10296.
## 7 NRP, RiPP        46   44214  131611   68485.    64902. 19801.
## 8 NRP, Other        32   43141   99576   76136.    78536. 20579.
## 9 NRP, Polyketide, RiPP  21   58186  257631  124101.    91425  63076.
## 10 NRP, Other, Polyketide  18   47242  154012   80890.    73720. 33500.
## # i 12 more rows
```

How many BGCs in each category? (counting hybrids of categories as separate)

Here are the numbers

How many BGCs in each category? (lumping all hybrids into one except NRPS-PKS)

```
# Lump any hybrid category with fewer than 80 BGCs into an "all other" category
# - Threshold determined arbitrarily to improve visualization
region_summary_lumped <- region_summary %>%
  mutate(
    group = if_else(n < 80, "All other hybrids", cats_str),
    group = group %>% fct_reorder(n)
  )

# Keep a reference DF handy for which categories got lumped
lump_groups <- region_summary_lumped %>% select(cats_str, group)

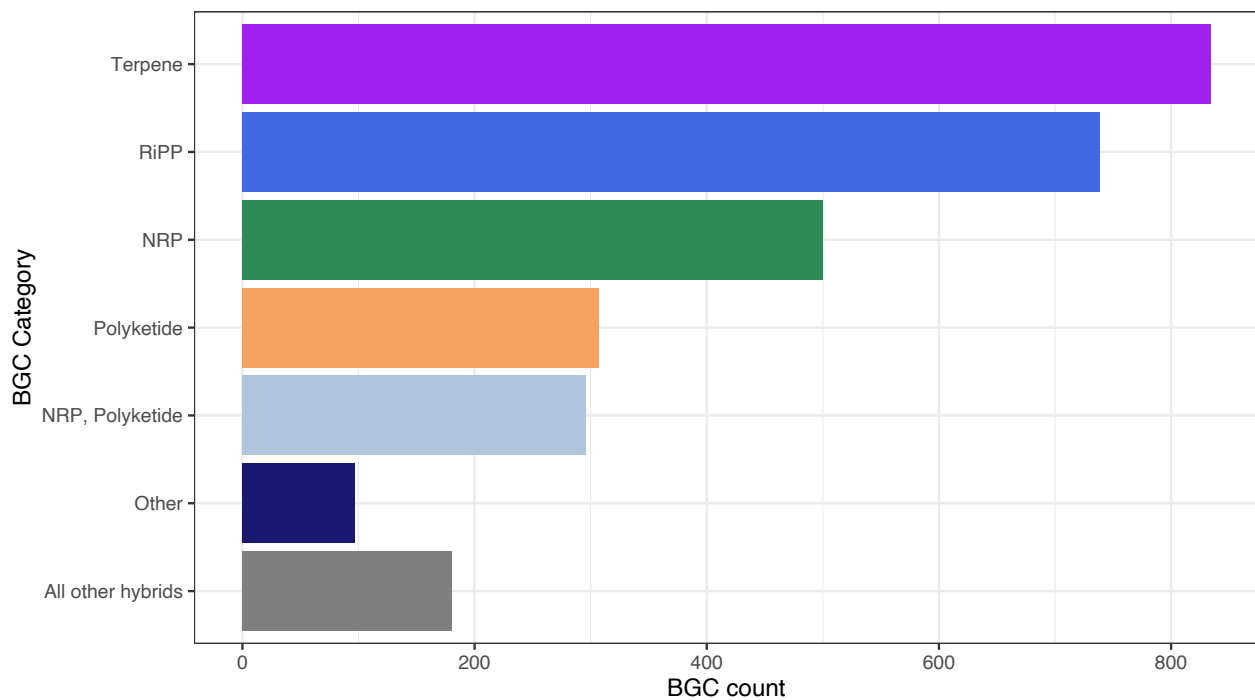
# Use the MIBiG / antiSMASH coloring scheme
cat_colors <- c(
  "Polyketide" = "#f4a460",
  "NRP" = "#2e8b57",
  "RiPP" = "#4169e1",
  "Terpene" = "purple",
```

```

"Saccharide" = "#deb887",
"Other" = "#191970",
"NRP, Polyketide" = "lightsteelblue",
"All other hybrids" = "gray50"
)

# Plot it
lumped_category_counts <- region_summary_lumped %>%
  ggplot(aes(y = reorder(group, n))) +
  geom_col(aes(x = n, fill = group)) +
  scale_x_continuous(name = "BGC count", breaks = breaks_extended()) +
  scale_y_discrete(name = "BGC Category") +
  scale_fill_manual(values = cat_colors) +
  theme_bw() +
  guides(fill = "none")
lumped_category_counts

```



```

ggsave("./figs/svg/category_counts_lumped.svg", lumped_category_counts, device = "svg")
ggsave("./figs/png/category_counts_lumped.png", lumped_category_counts, device = "png")

```

How do BGCs vary in length by category (or combination of categories)?

Un-lumped categories

```

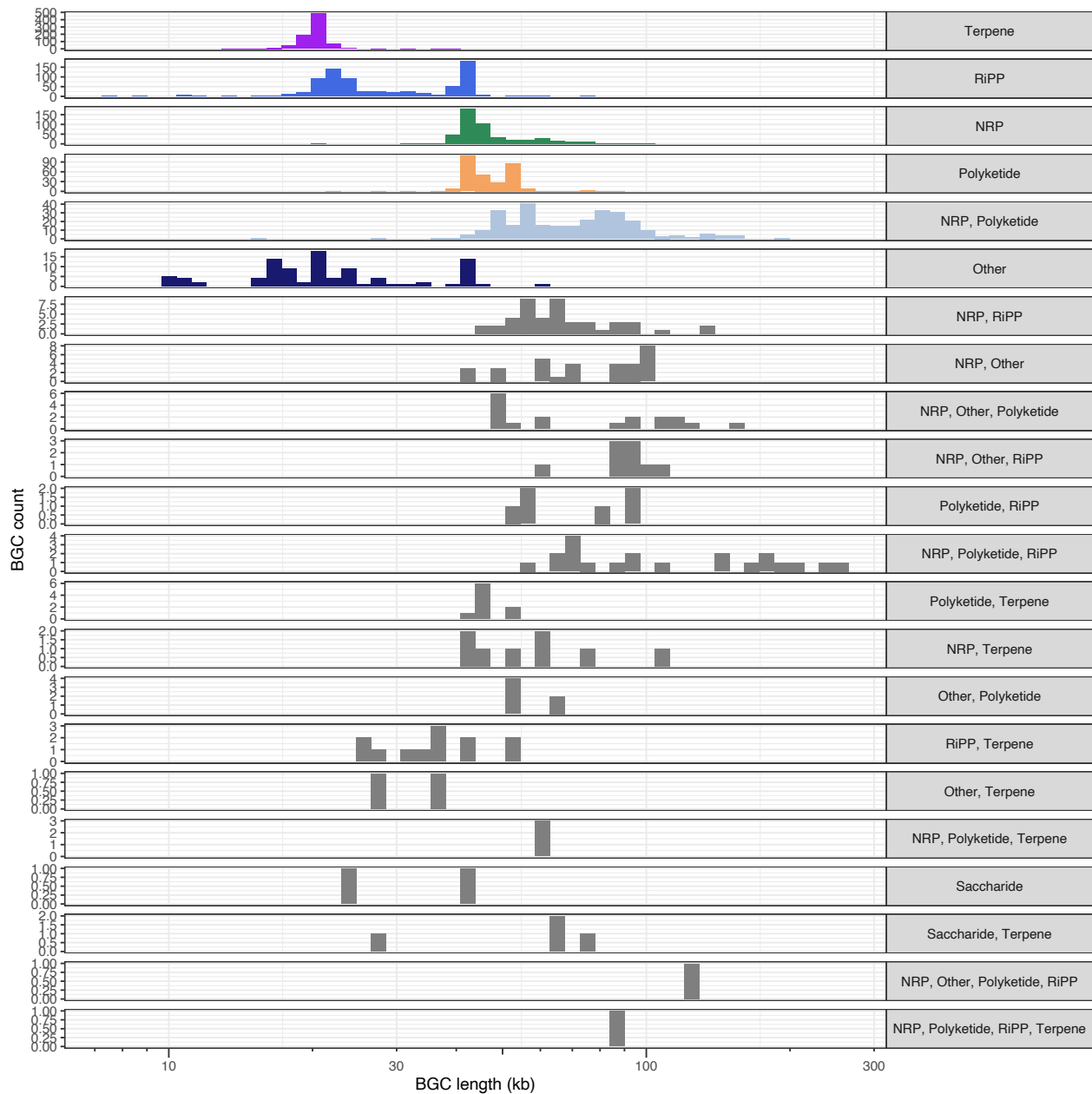
regions_lumped <- regions %>% left_join(lump_groups, by = "cats_str")

region_hist <- ggplot(regions_lumped, aes(
  x = region_length / 1000,
)) +

```

```
geom_histogram(aes(fill = group), bins = 50) +
scale_x_log10(name = "BGC length (kb)", guide = "axis_logticks", breaks = breaks_log(), labels = label_log()) +
scale_y_continuous(name = "BGC count", breaks = breaks_extended(), labels = label_comma()) +
scale_fill_manual(values = cat_colors) +
facet_grid(rows = vars(cats_str), scales = "free_y") +
theme_bw() +
theme(strip.text.y.right = element_text(angle = 0)) +
guides(fill = FALSE)
```

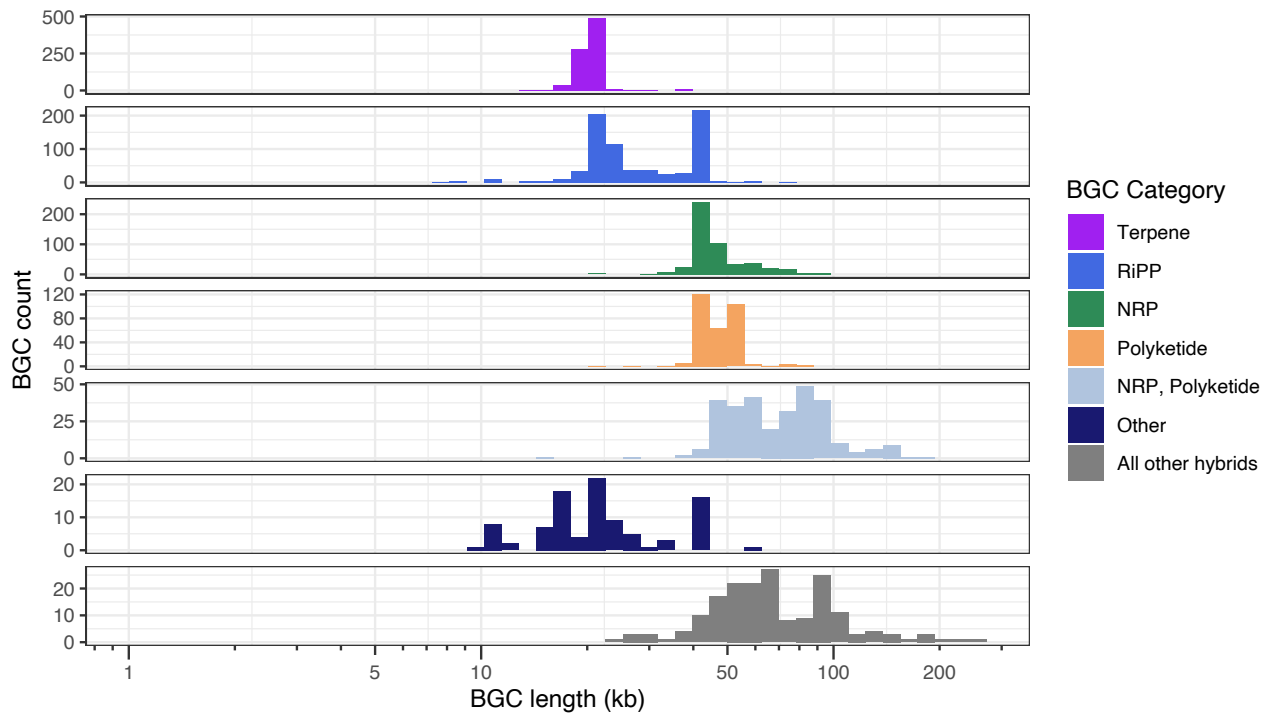
region_hist



```
ggsave("./figs/svg/region_hist.svg", region_hist, device = "svg")
ggsave("./figs/png/region_hist.png", region_hist, device = "png")
```

Lumped categories (again, except for NRPS-PKS hybrids)

```
region_hist_lumped <- regions_lumped %>%
  # filter(group != "All other hybrids") %>%
  ggplot(aes(x = region_length / 1000)) +
  geom_histogram(aes(fill = group), bins = 50) +
  scale_x_log10(name = "BGC length (kb)", guide = "axis_logticks", limits = c(1, NA), breaks = c(1, 5, 10, 50, 100, 200)) +
  scale_y_continuous(name = "BGC count", breaks = breaks_extended(n = 3)) +
  scale_fill_manual(values = cat_colors) +
  facet_wrap(vars(group), ncol = 1, scales = "free_y") +
  guides(fill = guide_legend(title = "BGC Category")) +
  theme_bw() +
  theme(
    strip.background = element_blank(),
    strip.text = element_blank()
  )
region_hist_lumped
```



```
ggsave("./figs/svg/region_hist_lumped.svg", region_hist_lumped, device = "svg")
ggsave("./figs/png/region_hist_lumped.png", region_hist_lumped, device = "png")
```

How does BGC count vary across genera and by category?

Table:

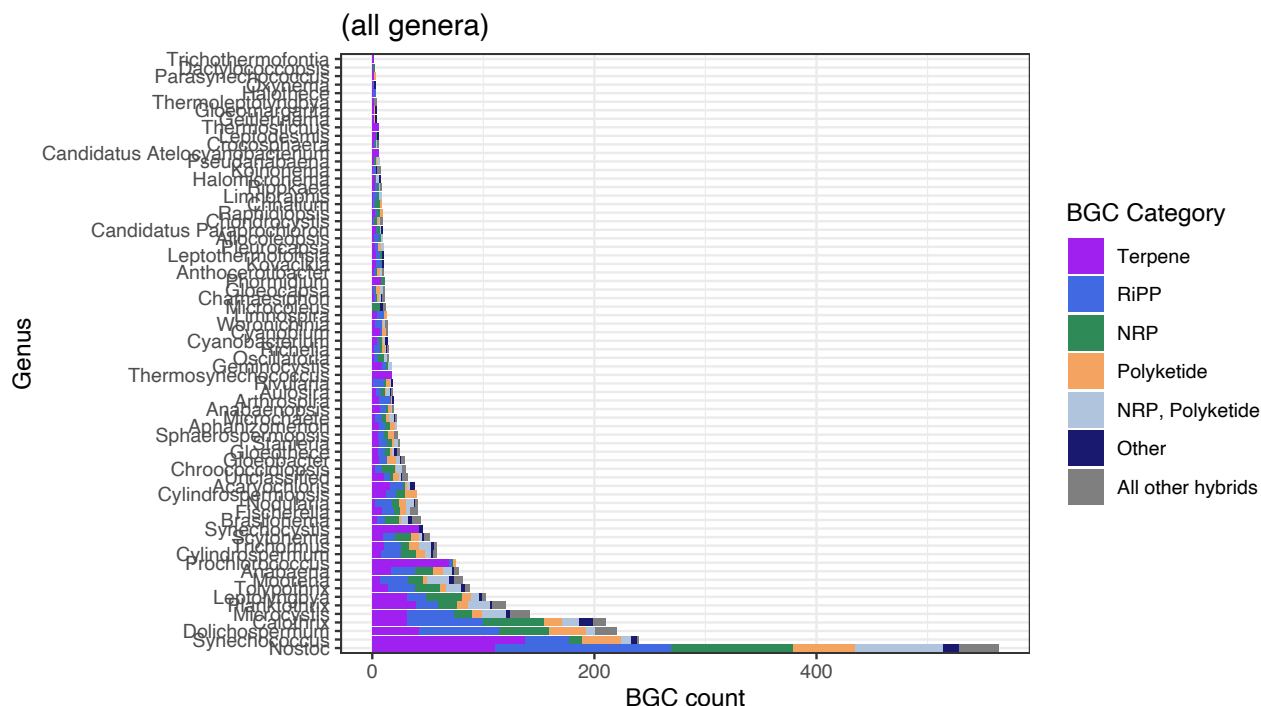
```
tax_count <- regions_lumped %>%
  group_by(tax_genus, cats_str) %>%
  summarize(num_bgcs = n()) %>%
  mutate(tax_genus = tax_genus %>% fct_reorder(num_bgcs)) %>%
  left_join(lump_groups, by = "cats_str") # %>%
```

```
tax_count
```

```
## # A tibble: 388 x 4
## # Groups:   tax_genus [70]
##   tax_genus      cats_str      num_bgcs group
##   <fct>         <fct>         <int> <fct>
## 1 Acaryochloris Terpene           16 Terpene
## 2 Acaryochloris RiPP             12 RiPP
## 3 Acaryochloris NRP              2 NRP
## 4 Acaryochloris Polyketide        2 Polyketide
## 5 Acaryochloris NRP, Polyketide    2 NRP, Polyketide
## 6 Acaryochloris Other             4 Other
## 7 Allocatedopsis Terpene           2 Terpene
## 8 Allocatedopsis RiPP             4 RiPP
## 9 Allocatedopsis NRP              2 NRP
## 10 Allocatedopsis NRP, Polyketide  1 NRP, Polyketide
## # i 378 more rows
```

Raw BGC counts by genus

```
p_all <- tax_count %>%
  group_by(tax_genus) %>%
  filter(sum(num_bgcs) > 0) %>%
  ggplot(aes(x = fct_infreq(tax_genus, w = num_bgcs))) +
  geom_col(aes(y = num_bgcs, fill = group), position = position_stack(reverse = TRUE)) +
  scale_y_continuous(name = "BGC count", breaks = breaks_extended()) +
  scale_x_discrete(name = "Genus") +
  scale_fill_manual(name = "BGC Category", values = cat_colors) +
  coord_flip() +
  theme_bw() +
  ggtitle("(all genera)")
p_all
```



```
ggsave("./figs/svg/genus_counts_all.svg", p_all, device = "svg")
ggsave("./figs/png/genus_counts_all.png", p_all, device = "png")
```

In order to normalize BGC counts to a per-genome basis, we must also know how many Cyano genomes are in SMC but *lack* BGCs (as detected by antiSMASH).

```
# Plaintext file listing all the accessions that had no BGCs
cyano_nohits <- read_tsv("data/ncbi_cyano_nohit_accs.txt", col_names = c("accession_id")) %>%
  left_join(cyano_asm_tax, by = join_by(accession_id == assembly_accession)) %>%
  filter(accession_id %in% ncbi_hiq_meta$`Assembly Accession`)

# Incorporate these into our genome counts
cyano_nohit_genus_counts <- cyano_nohits %>%
  group_by(genus) %>%
  summarize(n_nohits = n()) %>%
  mutate(genus = replace_na(genus, "Unclassified")) %>%
  arrange(genus)

genomes_by_genus <- read_tsv("data/2025-02-25-1442-cyano_smc_src_counts_by_genus.tsv")
genomes_by_genus <- genomes_by_genus %>%
  left_join(cyano_nohit_genus_counts, by = join_by(tax_genus == genus)) %>%
  mutate(n_nohits = replace_na(n_nohits, 0)) %>%
  mutate(tot_genomes = n_nohits + n_sources, .keep = "unused")

hiq_genomes_by_genus <- ncbi_hiq_meta %>%
  semi_join(smc_tracked_cyanos_df, by = join_by(`Assembly Accession` == accession)) %>%
  left_join(cyano_asm_tax, by = join_by(`Assembly Accession` == assembly_accession)) %>%
  group_by(genus) %>%
  summarize(tot_genomes = n()) %>%
  rename(tax_genus = genus)
```

Prepare a single data frame to use for the plots

```
df_plots <- tax_count %>%
  filter(tax_genus != "Unclassified") %>%
  left_join(hiq_genomes_by_genus, by = "tax_genus") %>%
  mutate(bgc_dens = num_bgcs / tot_genomes) %>%
  group_by(tax_genus) %>%
  mutate(tot_bgc_dens = sum(bgc_dens), tot_bgcs = sum(num_bgcs))
```

Plot the BGCs per genome (normalized to 100%) alongside number of BGCs per genus and genomes per genus

```
# Plot BGCs per genome, colored by category and divided by genus
p_bgc_dens <- df_plots %>%
  ggplot(aes(x = fct_rev(tax_genus))) +
  geom_col(aes(y = bgc_dens, fill = group), position = position_fill(reverse = TRUE)) +
  scale_y_continuous(name = "BGC proportion") +
  scale_x_discrete(name = "Genus") +
  scale_fill_manual(name = "BGC Category", values = cat_colors) +
  coord_flip() +
  theme_bw() +
  theme(legend.position = "bottom")

# Plot total BGC count by genus
p_bgc_ct <- df_plots %>%
  ggplot(aes(x = fct_rev(tax_genus))) +
  geom_col(aes(y = tot_bgcs), data = df_plots %>% select(tax_genus, tot_bgcs, tot_bgc_dens) %>% distinct()) +
  scale_y_continuous(
    name = "BGC count",
    trans = transform_pseudo_log(base = 10),
    breaks = c(0, 1, 5, 10, 20, 50, 100, 200, 500)
  ) +
  coord_flip() +
  theme_bw() +
  theme(
    axis.title.y = element_blank(),
    axis.text.y = element_blank()
  )

# Plot genome count by genus
genome_counts <- df_plots %>%
  group_by(tax_genus, tot_genomes) %>%
  summarize(tot_bgc_dens = sum(bgc_dens), tot_bgcs = sum(num_bgcs)) %>%
  arrange(tot_bgc_dens)
genome_counts
```

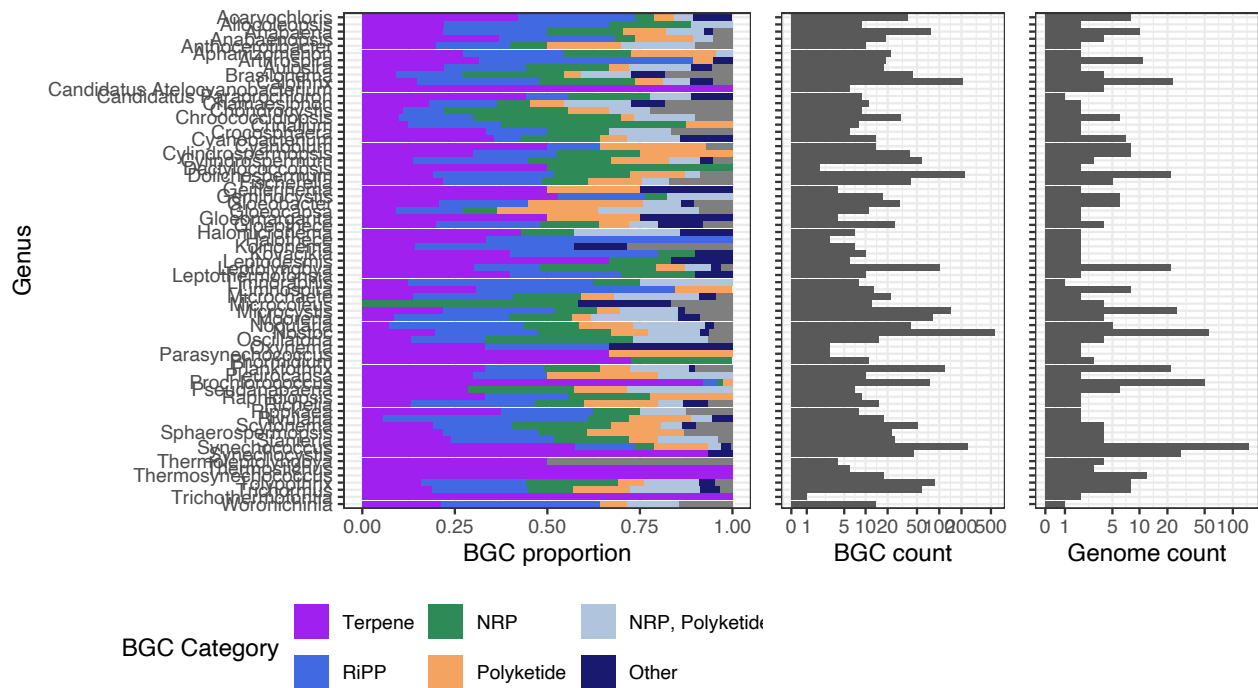
```
## # A tibble: 69 x 4
## # Groups:   tax_genus [69]
##   tax_genus tot_genomes tot_bgc_dens tot_bgcs
##   <chr>         <int>         <dbl>    <int>
## 1 Trichothermofontia      2         0.5      1
## 2 Dactylococcopsis       2         1       2
## 3 Thermoleptolyngbya     4         1       4
```

```
## 4 Pseudanabaena 6 1.17 7
## 5 Candidatus Atelocyanobacterium 4 1.5 6
## 6 Halothece 2 1.5 3
## 7 Oxynema 2 1.5 3
## 8 Parasynechococcus 2 1.5 3
## 9 Prochlorococcus 50 1.5 75
## 10 Thermosynechococcus 12 1.5 18
## # i 59 more rows
```

```
p_genome_ct <- genome_counts %>%
  ggplot(aes(x = fct_rev(tax_genus))) +
  geom_col(aes(y = tot_genomes)) +
  scale_y_continuous(
    name = "Genome count",
    trans = transform_pseudo_log(base = 10),
    breaks = c(0, 1, 5, 10, 20, 50, 100, 200, 500, 1000)
  ) +
  coord_flip() +
  theme_bw() +
  theme(
    axis.title.y = element_blank(),
    axis.text.y = element_blank()
  )
```

Plot them all together

```
p4 <- plot_grid(p_bgc_dens, p_bgc_ct, p_genome_ct, align = "h", rel_widths = c(3, 1, 1), nrow = 1)
p4
```



```
ggsave("./figs/png/split_proportional.png", p4, width = 8, height = 11, device = "png")
```

Prepare the GCF data


```
gcf_df <- bigslice_df %>%
  semi_join(regions, by = join_by(orig_folder == accession_id)) %>%
  select(genus, gcf_id) %>%
  distinct() %>%
  count(genus, name = "n_gcfs") %>% arrange(genus)
```

Plot the BGCs per genome (not normalized to 100%) alongside number of BGCs per genus, number of GCFs per genus, and genomes per genus

```
text_y_off <- theme(
  axis.title.y = element_blank(),
  axis.text.y = element_blank()
)

# Specific ordering for genus names to enforce for other plots
genus_order <- df_plots %>%
  ungroup() %>%
  mutate(tax_genus = fct_reorder(tax_genus, tot_bgc_dens, .desc = T)) %>%
  pull(tax_genus) %>%
  levels()

# Plot BGCs per genome, colored by category and divided by genus
p_bgc_dens <- df_plots %>%
  ggplot(aes(x = fct_relevel(tax_genus, genus_order))) +
  geom_col(aes(y = bgc_dens, fill = group), position = position_stack(reverse = TRUE)) +
  scale_y_continuous(name = "BGCs per genome", expand = expansion(add = c(0.1, 1))) +
  scale_x_discrete(name = "Genus") +
  scale_fill_manual(
    name = "BGC Category",
    values = cat_colors,
    guide = guide_legend(position = "inside")
  ) +
  coord_flip() +
  theme_bw() +
  theme(
    legend.title = element_text(face = "bold", size = rel(0.6)),
    legend.key.height = unit(0.25, "cm"),
    legend.key.width = unit(0.25, "cm"),
    legend.justification.inside = c(0.98, 0.99),
    axis.text.y = element_text(face = "italic", size = rel(0.8), vjust = 0.5)
  )

# Plot total BGC count by genus
p_bgc_ct <- df_plots %>%
  ggplot(aes(x = fct_relevel(tax_genus, genus_order))) +
  geom_col(aes(y = tot_bgcs), data = df_plots %>% select(tax_genus, tot_bgcs, tot_bgc_dens) %>% distinct()) +
  scale_y_continuous(
    name = "BGC count",
    trans = transform_pseudo_log(base = 10),
    breaks = c(0, 1, 5, 10, 100, 500)
  ) +
  coord_flip() +
  theme_bw()
```

```

p_genome_ct <- genome_counts %>%
  ggplot(aes(x = fct_relevel(tax_genus, genus_order))) +
  geom_col(aes(y = tot_genomes)) +
  scale_y_continuous(name = "Genome count", trans = scales::transform_pseudo_log(base = 10), breaks = c(
  coord_flip() +
  theme_bw()

p_gcf_ct <- gcf_df %>%
  filter(genus %in% genome_counts$tax_genus) %>%
  ggplot(aes(x = fct_relevel(genus, genus_order))) +
  geom_col(aes(y = n_gcfs)) +
  scale_y_continuous(name = "GCF count", trans = scales::transform_pseudo_log(base = 10), breaks = c(0,
  coord_flip() +
  theme_bw()

# Plot them all together
p5 <- plot_grid(
  p_bgc_dens,
  p_bgc_ct + text_y_off,
  p_gcf_ct + text_y_off,
  p_genome_ct + text_y_off,
  align = "h",
  rel_widths = c(3, 1, 1, 1),
  nrow = 1
)
df_plots

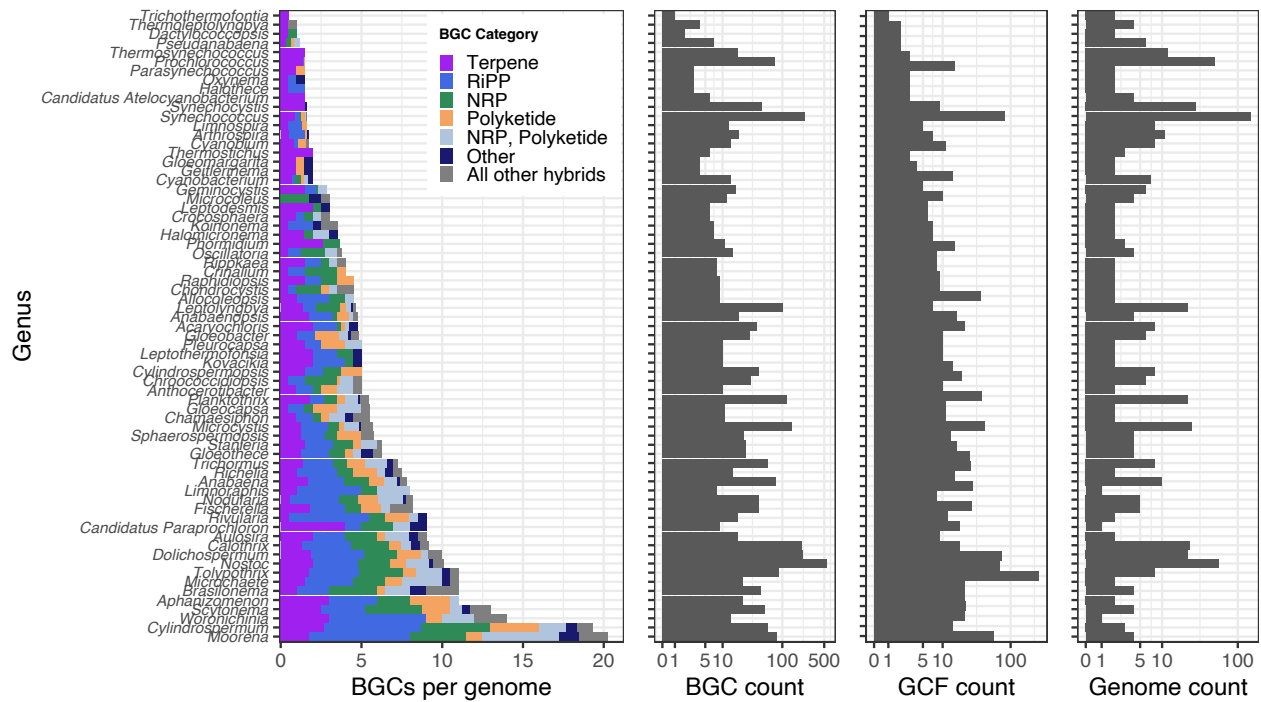
```

```

## # A tibble: 378 x 8
## # Groups:   tax_genus [69]
##   tax_genus cats_str num_bgcs group tot_genomes bgc_dens tot_bgc_dens tot_bgcs
##   <chr>      <fct>      <int> <fct>      <int>      <dbl>      <dbl>      <int>
## 1 Acaryochl~ Terpene      16 Terp~         8         2         4.75       38
## 2 Acaryochl~ RiPP        12 RiPP         8         1.5       4.75       38
## 3 Acaryochl~ NRP          2 NRP          8         0.25      4.75       38
## 4 Acaryochl~ Polyket~      2 Poly~         8         0.25      4.75       38
## 5 Acaryochl~ NRP, Po~      2 NRP,~         8         0.25      4.75       38
## 6 Acaryochl~ Other         4 Other         8         0.5       4.75       38
## 7 Allocoleo~ Terpene      2 Terp~         2         1         4.5        9
## 8 Allocoleo~ RiPP         4 RiPP         2         2         4.5        9
## 9 Allocoleo~ NRP          2 NRP          2         1         4.5        9
## 10 Allocoleo~ NRP, Po~      1 NRP,~         2         0.5       4.5        9
## # i 368 more rows

```

p5



```
ggsave("./figs/png/p_bgc_ct.png", p_bgc_ct, width = 8, height = 11, device = "png")
ggsave("./figs/png/p_genome_ct.png", p_genome_ct, width = 8, height = 11, device = "png")
ggsave("./figs/png/p_gcf_ct.png", p_gcf_ct, width = 8, height = 11, device = "png")
ggsave("./figs/png/split_triple.png", p5, width = 8, height = 11, device = "png")
```

Create Figure 2 for the manuscript

```
sm_margin <- unit(c(0.05, 0.05, 0.05, 0.05), "cm")

p_a <- plot_grid(
  p_bgc_dens + theme(
    legend.title = element_text(face = "bold", size = rel(0.8)),
    legend.key.label = element_text(size = rel(0.8)),
    legend.key.height = unit(0.25, "cm"),
    legend.key.width = unit(0.25, "cm"),
    legend.justification.inside = c(0.98, 0.99),
    axis.text.y = element_text(face = "italic", size = rel(0.8), vjust = 0.5),
    axis.text.x = element_text(size = rel(0.8)),
    plot.margin = sm_margin
  ),
  p_bgc_ct + text_y_off + theme(
    axis.text.x = element_text(size = rel(0.8)),
    axis.ticks.y = element_blank(),
    plot.margin = sm_margin
  ),
  p_gcf_ct + text_y_off + theme(
    axis.text.x = element_text(size = rel(0.8)),
    axis.ticks.y = element_blank(),
    plot.margin = sm_margin
  )
)
```

```

    plot.margin = sm_margin
  ),
  p_genome_ct + text_y_off + theme(
    axis.text.x = element_text(size = rel(0.8)),
    plot.background = element_rect(fill = NA),
    axis.ticks.y = element_blank(),
    plot.margin = unit(c(0.05, 0.15, 0.05, 0.05), "cm")
  ),
  align = "h",
  rel_widths = c(3, 1, 1, 1),
  nrow = 1
)

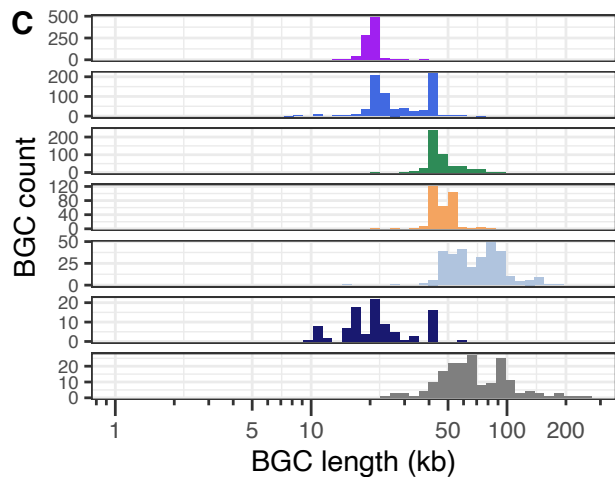
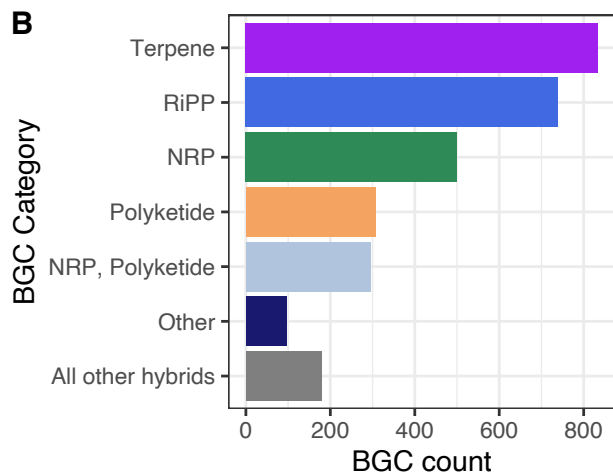
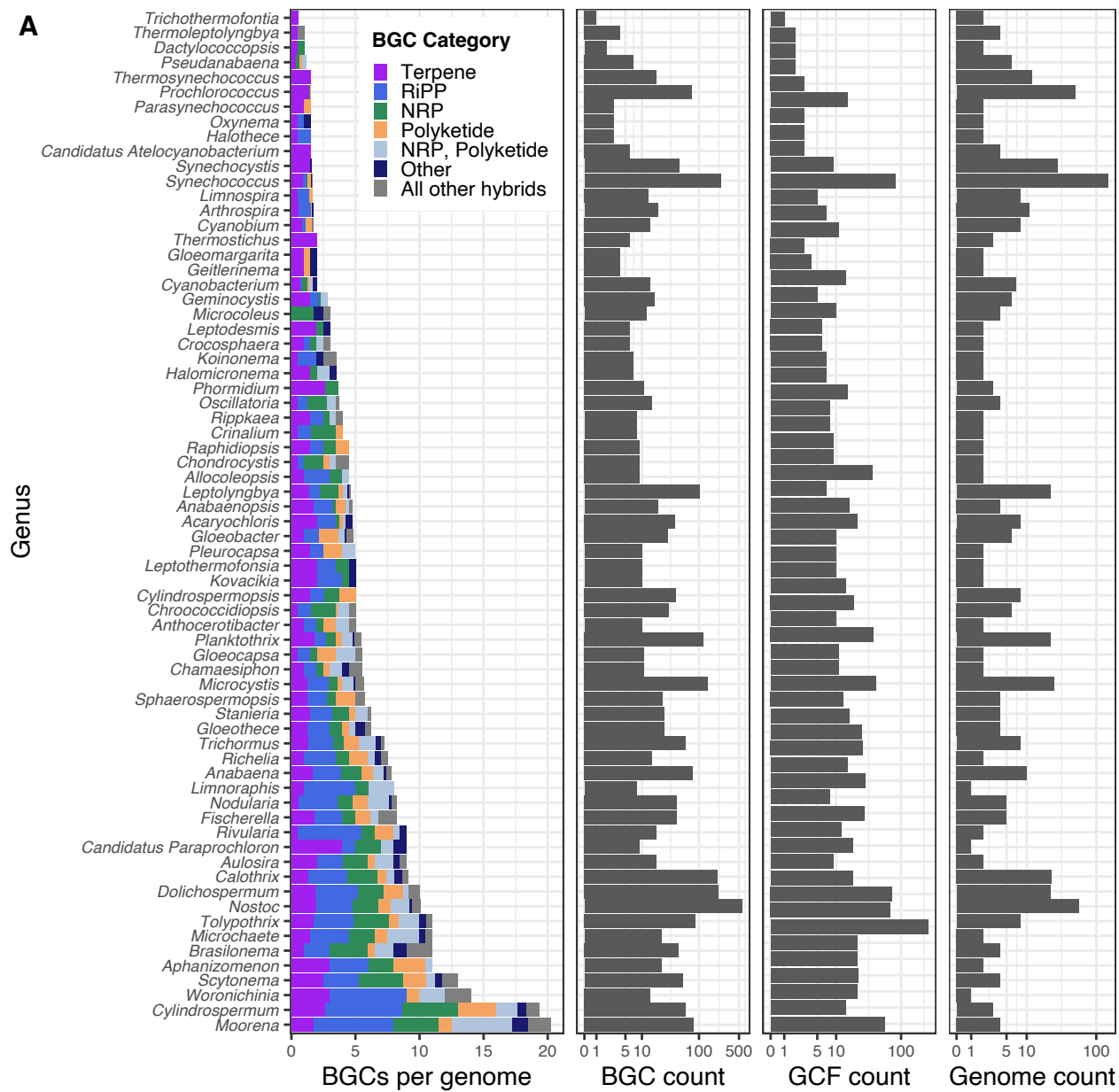
p_b <- ggplot(
  region_summary_lumped,
  aes(y = reorder(group, n))
) +
  geom_col(aes(x = n, fill = group)) +
  scale_x_continuous(name = "BGC count", breaks = breaks_extended()) +
  scale_y_discrete(name = "BGC Category") +
  scale_fill_manual(values = cat_colors) +
  theme_bw() +
  guides(fill = "none")

p_c <- regions_lumped %>%
  ggplot(aes(x = region_length / 1000)) +
  geom_histogram(aes(fill = group), bins = 50) +
  scale_x_log10(
    name = "BGC length (kb)",
    guide = "axis_logticks",
    limits = c(1, NA),
    breaks = c(1, 5, 10, 50, 100, 200)
  ) +
  scale_y_continuous(name = "BGC count", breaks = breaks_extended(n = 3)) +
  scale_fill_manual(values = cat_colors) +
  facet_wrap(vars(group), ncol = 1, scales = "free_y") +
  guides(fill = "none") +
  theme_bw() +
  theme(
    strip.background = element_blank(),
    strip.text = element_blank(),
    axis.text.y = element_text(size = rel(0.8)),
    panel.spacing = unit(0.25, "lines")
  )

bottom_row <- plot_grid(p_b, p_c, nrow = 1, labels = c("B", "C"), label_size = 12)

fig2 <- plot_grid(p_a, bottom_row, nrow = 2, labels = c("A", ""), label_size = 12, rel_heights = c(2.5,
fig2

```



```
ggsave("./figs/_fig2.png", fig2, width = 6.73, height = 9.17, units = "in", device = "png")
ggsave("./figs/_fig2.pdf", fig2, width = 6.73, height = 9.17, units = "in", device = "pdf")
```

Updates

What HiQ genomes are missing from the tracker?

```
setdiff(ncbi_hiq_meta$`Assembly Accession`, smc_tracked_cyanos_df$accession)
```

```
## [1] "GCA_032163395.1" "GCF_032163395.1" "GCA_047276495.1" "GCF_047276495.1"
## [5] "GCA_029953635.1" "GCF_029953635.1" "GCA_038050375.1" "GCF_038050375.1"
## [9] "GCA_963920485.1" "GCF_963920485.1" "GCA_003555505.2" "GCA_000014265.1"
## [13] "GCF_000014265.1" "GCA_040564635.1" "GCF_040564635.1" "GCA_042465725.1"
## [17] "GCF_042465725.1" "GCA_029910235.1" "GCF_029910235.1" "GCA_030323025.1"
## [21] "GCF_030323025.1" "GCA_029919255.1" "GCF_029919255.1" "GCA_030409055.1"
## [25] "GCF_030409055.1" "GCA_904830955.1" "GCF_904830955.1" "GCA_043099315.1"
## [29] "GCF_043099315.1" "GCA_046846445.1" "GCF_046846445.1" "GCA_046847935.1"
## [33] "GCF_046847935.1" "GCA_046847945.1" "GCF_046847945.1" "GCA_046847955.1"
## [37] "GCF_046847955.1" "GCA_030553035.1" "GCF_030553035.1" "GCA_041506625.1"
## [41] "GCF_041506625.1" "GCA_031754835.1" "GCF_031754835.1" "GCA_038396575.1"
## [45] "GCF_038396575.1" "GCA_030544905.1" "GCF_030544905.1" "GCA_009498715.2"
## [49] "GCF_009498715.2" "GCA_036630075.1" "GCF_036630075.1" "GCA_036630085.1"
## [53] "GCF_036630085.1" "GCA_036630095.1" "GCF_036630095.1" "GCA_036630065.1"
## [57] "GCF_036630065.1" "GCA_036630105.1" "GCF_036630105.1" "GCA_036630115.1"
## [61] "GCF_036630115.1" "GCA_003846445.2" "GCF_003846445.2" "GCA_047497115.1"
## [65] "GCF_047497115.1" "GCA_047497125.1" "GCF_047497125.1" "GCA_047304675.1"
## [69] "GCF_047304675.1" "GCA_047302745.1" "GCF_047302745.1" "GCA_047302905.1"
## [73] "GCF_047302905.1" "GCA_047304605.1" "GCF_047304605.1" "GCA_047302765.1"
## [77] "GCF_047302765.1" "GCA_047302795.1" "GCF_047302795.1" "GCA_047305125.1"
## [81] "GCF_047305125.1" "GCA_047305225.1" "GCF_047305225.1" "GCA_047302855.1"
## [85] "GCF_047302855.1" "GCA_047302875.1" "GCF_047302875.1" "GCA_043099305.1"
## [89] "GCF_043099305.1" "GCA_032620035.1" "GCF_032620035.1" "GCA_038630775.1"
## [93] "GCF_038630775.1" "GCA_035320825.1" "GCF_035320825.1" "GCA_033798705.1"
## [97] "GCF_033798705.1" "GCA_045345185.1" "GCF_045345185.1" "GCA_038096175.1"
## [101] "GCF_038096175.1" "GCA_043099735.1" "GCF_043099735.1" "GCA_043099725.1"
## [105] "GCF_043099725.1" "GCA_032393985.1" "GCF_032393985.1" "GCA_047649235.1"
## [109] "GCF_047649235.1" "GCA_047275885.1" "GCF_047275885.1" "GCA_044923475.1"
## [113] "GCF_044923475.1" "GCA_963665995.1" "GCF_963665995.1" "GCA_036345015.1"
## [117] "GCF_036345015.1" "GCA_040436315.1" "GCF_040436315.1" "GCA_963920495.1"
## [121] "GCF_963920495.1" "GCA_047303025.1" "GCF_047303025.1" "GCA_963920475.1"
## [125] "GCF_963920475.1" "GCA_047497135.1" "GCF_047497135.1" "GCA_963920505.1"
## [129] "GCF_963920505.1" "GCA_047304715.1" "GCF_047304715.1" "GCA_963920465.1"
## [133] "GCF_963920465.1" "GCA_047304545.1" "GCF_047304545.1" "GCA_047304065.1"
## [137] "GCF_047304065.1" "GCA_047303035.1" "GCF_047303035.1" "GCA_047302865.1"
## [141] "GCF_047302865.1" "GCA_047302755.1" "GCF_047302755.1" "GCA_047303495.1"
## [145] "GCF_047303495.1" "GCA_030518695.1" "GCF_030518695.1" "GCA_047303465.1"
## [149] "GCF_047303465.1" "GCA_031583225.1" "GCF_031583225.1" "GCA_031583205.1"
## [153] "GCF_031583205.1" "GCA_047302985.1" "GCF_047302985.1" "GCA_047302805.1"
## [157] "GCF_047302805.1" "GCA_030518735.1" "GCF_030518735.1" "GCA_031583475.1"
## [161] "GCF_031583475.1" "GCA_030518715.1" "GCF_030518715.1" "GCA_031583535.1"
## [165] "GCF_031583535.1" "GCA_030322965.1" "GCF_030322965.1" "GCA_031583555.1"
```

[169] "GCF_031583555.1" "GCA_031583495.1" "GCF_031583495.1" "GCA_031583455.1"
[173] "GCF_031583455.1" "GCA_031583435.1" "GCF_031583435.1" "GCA_031583415.1"
[177] "GCF_031583415.1" "GCA_031583395.1" "GCF_031583395.1" "GCA_047716785.1"
[181] "GCF_047716785.1" "GCA_031583515.1" "GCF_031583515.1" "GCA_031583255.1"
[185] "GCF_031583255.1" "GCA_031586415.1" "GCF_031586415.1" "GCA_031583185.1"
[189] "GCF_031583185.1" "GCA_030323045.1" "GCF_030323045.1" "GCA_030322985.1"
[193] "GCF_030322985.1" "GCA_031583375.1" "GCF_031583375.1" "GCA_031583335.1"
[197] "GCF_031583335.1" "GCA_031583355.1" "GCF_031583355.1" "GCA_031583295.1"
[201] "GCF_031583295.1" "GCA_031583315.1" "GCF_031583315.1" "GCA_047304175.1"
[205] "GCF_047304175.1" "GCA_047716815.1" "GCF_047716815.1" "GCA_047302785.1"
[209] "GCF_047302785.1" "GCA_047303165.1" "GCF_047303165.1" "GCA_047303345.1"
[213] "GCF_047303345.1" "GCA_047304505.1" "GCF_047304505.1" "GCA_047303155.1"
[217] "GCF_047303155.1" "GCA_047304725.1" "GCF_047304725.1" "GCA_047302735.1"
[221] "GCF_047302735.1" "GCA_047303415.1" "GCF_047303415.1" "GCA_047304785.1"
[225] "GCF_047304785.1" "GCA_047304495.1" "GCF_047304495.1" "GCA_047303045.1"
[229] "GCF_047303045.1" "GCA_047304515.1" "GCF_047304515.1" "GCA_047304905.1"
[233] "GCF_047304905.1" "GCA_047304705.1" "GCF_047304705.1" "GCA_047304555.1"
[237] "GCF_047304555.1" "GCA_047304245.1" "GCF_047304245.1" "GCA_047716805.1"
[241] "GCF_047716805.1" "GCA_034092395.1" "GCF_034092395.1" "GCA_047305155.1"
[245] "GCF_047305155.1" "GCA_040371545.1" "GCF_040371545.1" "GCA_047305235.1"
[249] "GCF_047305235.1" "GCA_034092415.1" "GCF_034092415.1" "GCA_034092375.1"
[253] "GCF_034092375.1" "GCA_034092465.1" "GCF_034092465.1" "GCA_047303175.1"
[257] "GCF_047303175.1" "GCA_047304615.1" "GCF_047304615.1" "GCA_047304625.1"
[261] "GCF_047304625.1" "GCA_047302975.1" "GCF_047302975.1" "GCA_047305135.1"
[265] "GCF_047305135.1" "GCA_047304685.1" "GCF_047304685.1" "GCA_034093315.1"
[269] "GCF_034093315.1" "GCA_047303505.1" "GCF_047303505.1" "GCA_047303555.1"
[273] "GCF_047303555.1" "GCA_047305115.1" "GCF_047305115.1" "GCA_047304535.1"
[277] "GCF_047304535.1" "GCA_047302725.1" "GCF_047302725.1" "GCA_047305145.1"
[281] "GCF_047305145.1" "GCA_047304525.1" "GCF_047304525.1" "GCA_047305015.1"
[285] "GCF_047305015.1" "GCA_047305065.1" "GCF_047305065.1" "GCA_032701645.1"
[289] "GCF_032701645.1" "GCA_000340565.3" "GCA_904830945.1" "GCF_904830945.1"
[293] "GCA_032813545.1" "GCF_032813545.1" "GCA_042134685.1" "GCF_042134685.1"
[297] "GCA_029636065.1" "GCF_029636065.1" "GCA_904830885.1" "GCF_904830885.1"
[301] "GCA_047302995.1" "GCA_026000695.2" "GCA_042083005.1" "GCA_037094475.1"
[305] "GCA_963665115.1" "GCA_042692285.1" "GCA_041937185.1" "GCA_038087005.1"
[309] "GCA_038087035.1" "GCA_042465645.1" "GCA_042919575.1" "GCA_963718615.1"
[313] "GCA_963706595.1" "GCA_035193785.1" "GCA_963702855.1" "GCA_963703315.1"
[317] "GCA_041937175.1" "GCA_041879365.1" "GCA_030759595.1" "GCA_041879355.1"
[321] "GCA_045159875.1" "GCA_041530395.1" "GCF_003990665.1" "GCA_047302775.1"
[325] "GCF_047302775.1" "GCA_037023215.1" "GCA_046116335.1" "GCA_963921295.3"
[329] "GCA_963921275.3" "GCA_037127495.1"